Dynamic User Steering in Overlapping Next-Generation ORAN Cells Using Deep Q-Learning

Guilherme Claro*†, Pedro Valente*†, Duarte Raposo*, Pedro Rito*†,
Miguel Luís*‡, Pedro Santos§, Susana Sargento*†
*Instituto de Telecomunicações, 3810-193 Aveiro, Portugal
†DETI, University of Aveiro, 3810-193 Aveiro, Portugal
‡Instituto Superior Tecnico, University of Lisbon, Av. Rovisco Pais 1, Lisboa 1049-001, Portugal
§Faculdade de Engenharia, Universidade do Porto, Rua Roberto Frias, 4200-465 Porto, Portugal

***Abstract*—Cellular deployments in dense urban environments will face a growing number of User Equipments (UEs) and increasing data demand due to applications such as the tactile internet in next-generation cellular networks (6G). This trend will require the use of higher frequency bands, which reduce coverage. Consequently, operators will need to deploy multiple cells operating on different bands in close proximity, creating overlapping coverage. This overlap can lead to uneven loads and unbalanced networks when using traditional handover mechanisms based on Reference Signal Received Power (RSRP). In scenarios with many co-located cells and fluctuating demand, ORAN disaggregation, through the exposition of standard interfaces via the Near-Real-Time RAN Intelligent Controller (Near-RT RIC), enables proactive re-distribution of users to cells. This paper proposes a Deep-Q Learning (DQL) agent that steers users between overlapping antennas based on Key Performance Metrics (KPMs) telemetry data extracted directly from the RAN. The rewards aim to promote load fairness against mobility costs by penalising handovers. Actions are discrete re-associations of UE-Cells reassignments, and the state encodes the current UE to antenna mappings together with normalised antenna loads, so that mixed numerologies remain comparable. This approach is evaluated in a real testbed which runs on an open-source stack, and it is evaluated using real devices, specifically two antennas overlapped operating on different bands. Starting from an unbalanced initial condition, the agent enforces simulated handovers while aiming to preserve user Quality of Service (QoS) and relieve overloaded cells. Overall, the results show that learn-driven control provides telemetry load balancing and controlled handovers in dense smart city radio access networks.**

***Index Terms*—6G, ORAN, Near-RT RIC, xApp, Deep Q-Learning**

I. Introduction

Next-generation (6G) cellular deployments in dense urban environments are expected to accommodate a rapidly growing number of User Equipments (UEs) and unprecedented data demand from applications such as the tactile internet. This trend will require the use of higher frequency bands, which reduce coverage and necessitate the deployment of multiple cells operating on different bands in close proximity, creating overlapping coverage. Such ultra-dense deployments, characteristic of 5G infrastructures, aim to meet high-capacity and low-latency requirements but lead to frequent handovers and instability in handover management [1]. In urban environments, overlapping cell coverage and dense cell layouts exacerbate these issues, causing ping-pong handovers, signalling overhead, and service interruptions, showing that managing cellular connections based solely on signal quality metrics such as Reference Signal Received Power (RSRP) is no longer sufficient [2]. In scenarios with many co-located cells and fluctuating demand, Open Radio Access Network (ORAN) disaggregation, through the exposition of standard interfaces via the Near-Real-Time RIC (Near-RT RIC), enables proactive redistribution of users across cells.

In legacy Radio Access Network (RAN) implementations, base stations operate monolithically, using fixed, proprietary algorithms for scheduling, handover and load management, which cannot easily adapt to dynamic load variations or severe inter-cell interference [3]. This calls for a new architectural approach and control-loop algorithms that can provide intelligent and programmable control over the RAN [4]. The ORAN principles, and the introduction of a new control plane element, the RAN Intelligent Controller (RIC), address this need [5]. The Near-RT RIC provides a platform for deploying xApps, which in turn use detailed network telemetry to control the RAN in a closed-loop. This enables telemetry-driven control loops on the order of 10 ms, fast enough to respond to fluctuations in traffic demand or radio conditions in near real-time, promising improved spectrum efficiency and Quality of Service (QoS) in dense 5G scenarios.

This work is motivated by the convergence of these trends and challenges. By exploiting the Near-RT RIC's telemetry and control capabilities, a new level of multi-cell coordination and intelligence can be introduced, difficult to achieve in legacy cellular networks [2]. In this work, we propose and validate a closed-loop traffic steering solution based on Deep Q-Learning (DQL). A DQL agent is designed to use Near-RT RIC telemetry data for the management of User Equipment (UE) associations in overlapping cell scenarios. In contrast to traditional RSRP based handovers, a reward function is defined to balance load reasonably across Distributed Units (DUs) against the handover costs. Through

experimental validation in a controlled environment testbed, we demonstrate that a balanced steady state between the DUs is achieved, resulting in a significant reduction of the cell congestion, showing the feasibility of using a xApp-driven DQL agent for dynamic resource management. Even under a highly imbalanced scenario ($63\% - 0\%$ load split), the DQL agent was capable of achieving a balanced utilisation across the two cells.

The remainder of this paper is organised as follows. Section II reviews the related work, and section III describes the solution that is going to be implemented and its architecture. Section IV describes the DQL agent responsible for the traffic steering decisions. Section V presents and evaluates the results, and finally, section VI enumerates the conclusions and insights for future work.

II. Related Work

Reinforcement Learning (RL) has been explored for user association and load balancing in 5G and heterogeneous network (HetNet) scenarios. For instance, the work in [6] applies DQL to user association problems and reports improvements over the Maximum Signal-to-Interference-plus-Noise Ratio (SINR) approach in simulated environments. Complementary approaches incorporate QoS-aware RL-based handover and load-balancing mechanisms. In [7], the authors propose a QoS-aware variant of Clipped DQL that combines resource usage indicators with QoS metrics in the reward function to guide handover decisions. In the context of ORAN approaches, prior works have laid methodologies and tools for enabling ML-driven closed-loop control. The authors of [4] present a framework that supports the development and evaluation of ML-based xApps on programmable experimental O-RAN platforms. Similarly, [3] explores programmable steering mechanisms within ORAN architectures. More recently, the work in [8] demonstrates the feasibility of RL-enabled xApps deployed on O-RAN Software Community RIC (O-RAN SC RIC) integrated with srsRAN, validating end-to-end closed loop control in an open-source ecosystem.

In contrast to the works cited above, and unlike approaches relying solely on simulated SINR/QoS environments, this work introduces an ORAN Near-RT RIC traffic steering loop that leverages Key Performance Measurement (KPM) telemetry data collected from a real-world testbed employing actual UEs. We formulate a DQL agent that operates on normalised Physical Resource Block (PRB) load states (to ensure comparability across different bandwidths and numerologies), and employs a reward structure that explicitly balances load fairness against handover stability via penalties for handovers and ping-pong effects. Furthermore, we systematically quantify the sensitivity to exploration schedules, training budgets, and reward weighting hyperparameters in an overlapping-cell deployment.

III. ORAN Architecture Overview

This section describes the ORAN architecture and Near-RT RIC components used to realise the proposed closed-loop traffic-steering pipeline. The architecture, presented in

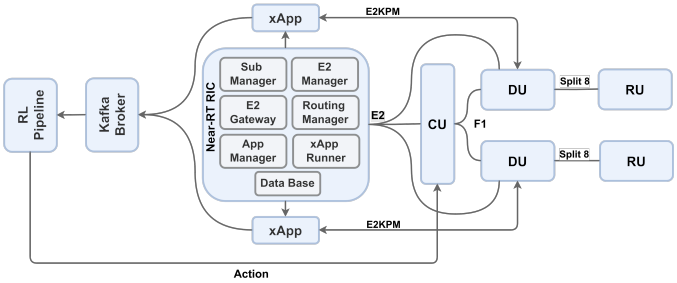


Fig. 1: Disaggregated RL-based user-steering ORAN architecture.

Figure 1, promotes a disaggregated RAN where the different functions are split across the three components: 1) the Centralised Unit (CU) that hosts the higher-layer functions; 2) the DU that is responsible for the radio control function; 3) and the Radio Unit (RU), responsible for implementing radio frequency processing. The CU-DU split is made through the F1 interface, which will enable intra-CU handovers, while the DU-RU connection uses the split 8.

The Near-RT RIC is implemented as a set of microservices that jointly handle messaging, state management, onboarding and protocol termination separately. The routing manager enables asynchronous communication between platform services and xApps. E2 Termination controls the E2 Application Protocol session with connected E2 nodes, while the Subscription Manager handles and controls E2 node subscriptions and xApp subscriptions to these nodes. The application manager and xApp runner oversee xApp lifecycle tasks, such as activation, configuration, and health checks. Finally, the Near-RT RIC also has a RAN information database that acts as the near-real-time data source for the RAN topology exposed by xApps.

As mentioned before, the Near-RT RIC hosts xApps that can subscribe to RAN measurements. These xApps will communicate with the RAN via the E2 interface, which will carry telemetry indications from the DUs to the Near-RT RIC by using E2 Service Model (E2SM), such as E2SM-KPM. E2SM-KPM obtains key performance measurements, e.g. the per-UE downlink and uplink PRB utilisation and throughput. It also has the E2SM-RAN Control (RC) that can deliver control messages from the Near-RT RIC to the RAN components when supported by the E2 agent. In particular, the xApps will extract E2-KPM periodical measurements from the DUs over the E2 interface to the Near-RT RIC, aggregate them, and send them to the RL pipeline, where the RL agent selects a steering decision.

The key performance metrics collected over the E2 interface provide the measurements required to construct the agent state. These metrics are aggregated into fixed-interval samples to be used by a RL pipeline to select traffic-steering actions. Training the agent will rely on experience tuples built from consecutive state samples and the applied actions (UE-to-DU associations).

IV. Reinforcement Learning for Resource Management

This work proposes a dynamic user steering solution based on RL to redistribute users and mitigate cell congestion using telemetry data from Near-RT RIC xApps. This section describes the proposed DQL pipeline, including the data flow, the reward function, and the overall training loop.

Metrics Aggregator: Two monitoring xApps subscribe to per-cell and per-UE KPM indicators on each DU in the overlapped area, and send JSON records with the main key identifier (the UE ID). Each record carries the source DU, controller timestamp, source timestamps, and a metrics set that includes the downlink and uplink UE throughput and per-UE PRB occupancy. An aggregator consumes this stream, de-duplicates late or malformed messages, and aligns samples onto a decision grid by interpolating or dropping samples outside the tolerance window. To stabilise decisions against short-term jitter, the aggregator computes a moving average of each UE's PRB usage over a 60 second window, using only the valid entries. When DUs operate with different band configurations, the same bandwidth utilisation may correspond to different PRBs occupied. This happens because the bandwidth represented by one PRB increases with the Subcarrier Spacing (SCS), making raw PRB-based load values non-comparable across cells. Each DU is configured with a SCS, denoted by Δf; since a PRB contains 12 subcarriers, its frequency span is $12\,\Delta f$. Therefore, the absolute bandwidth of a PRB increases with Δf. To compare the occupied loads fairly across mixed numerologies, each UE's demand is converted into occupied bandwidth at the target DU, and normalised by the DU's total capacity in kHz.

Reward Function: The proposed reward function, calculated every step t as r_t, is designed to balance load distribution across the DUs, providing a level of handover stability. Let $\mathbf{l}_t \in [0,1]^D$ be the vector of DU load percentages, where D is the number of DUs, observed after the action at step t. Every action is penalised, reflecting the impact of the resulting state on the network's efficiency and reliability as follows:

$$r_t = -\omega \cdot \text{StdDev}(\mathbf{l}_t) - \alpha \cdot h_t - \beta \cdot \text{pp}_t. \tag{1}$$

The weights ω, α, and β define the penalty function to discourage load imbalance and excessive handovers: ω is associated with the load imbalance between DUs, α weights the occurrence of handovers, indicated by the flag $h_t \in \{0,1\}$, and β weights the ping-pong events, triggered by the flag $\text{pp}_t \in \{0,1\}$ when a UE switches to a different DU, and returns to the previous one within two consecutive steps.

The action-value function $Q_\theta(s,a)$, where s is the state and a is the action, is approximated by a two-layer Multilayer Perceptron (MLP) with a hidden layer structure of 128–128 and ReLU activations. The non-linearity of ReLU enables the agent to learn decision boundaries, such as identifying specific load thresholds at which a handover becomes beneficial. Transitions (s_t, a_t, r_t, s_{t+1}) generated at each step t are stored in the replay buffer. During the optimisation, a batch B of transitions is sampled from the replay buffer, and we index each sampled transition by $i \in B$, i.e., (s_i, a_i, r_i, s_i').

$$y_i = r_i + \gamma \max_{a'} Q(s_i', a'; \theta^-), \tag{2}$$

Here s_i' denotes the next state of transition i, and y_i is the one-step Temporal Difference computed using the target network with parameters θ^-, which are copied from θ to stabilise learning.

A discount factor $\gamma = 0.9$ was chosen to balance immediate responsiveness with stability in near-real time control-loop. Smaller values would make the policy myopic, which would lead to unnecessary handovers, whereas larger values would overweight uncertain outcomes, which would slow convergence and make the learning unstable. $\gamma = 0.9$ has an effective horizon of $\frac{1}{1-\gamma} \approx 10$, which is closer to the time scale on which steering actions affect the measured loads. The network weights θ are updated by minimising the mean-squared Temporal Difference error over a batch B:

$$\mathcal{L}_{TD}(\theta) = \frac{1}{|B|} \sum_{i \in B} \Big(y_i - Q(s_i, a_i; \theta) \Big)^2. \tag{3}$$

Optimisation is performed using the Adam optimiser with a learning rate of 10^{-3}, which adapts the rate for each weight individually to accelerate convergence. Action selection follows an ε-greedy policy where ε is the probability of selecting a random action for exploration and decays over training according to the chosen schedule.

Control Loop: The complete control loop is detailed in Algorithm 1. A brief description of its operation process is given below:

- L.1-2: the Replay Buffer $\mathcal{D}$ and epsilon schedule are defined and the Q network is initialised.
- L.3: the metrics aggregator receives the snapshot of the average PRB usage of each UE.
- L.4-5: the initial state is received based on the demands and the assignments. Training loop begins.
- L.6-8: the agent converts the current assignments into One-Hot vectors, normalises the DU loads, and concatenates them into an input tensor vector.
- L.15-16: an action is executed that results in a new state and a reward associated with that action.
- L.17: the state, action, reward, and next state are stored in the Replay Buffer.
- L.18-20: agent draws a random sample from the buffer and adjusts the neural network weights by minimising Mean Squared Error (MSE) between prediction and target.
- L.21-23: the current state is updated at the end of each episode, and the ε decays gradually.

Finally, the steering task is formalised as a Markov Decision Process (MDP), represented in Figure 2. The state s_t is composed of (i) the current UE-DU associations and (ii) the corresponding normalised DU load levels derived from the telemetry stream produced by the Kafka consumer and aggregated over the decision window. The action a_t is either

Algorithm 1 Training Loop For Deep Q-Learning Agent

Input: Replay Buffer $\mathcal{D}$, Epsilon schedule
INITIALIZE_Q_NETWORK(θ)
$Demands \leftarrow Aggregator.GetRobustSnapshot()$
$State \leftarrow Environment.Get(Demands)$
for each $Episode$ from 1 to M **do**
  for each $Step$ t from 1 to T **do**
    $S_a \leftarrow$ OneHot($State.Assign$)
    $S_l \leftarrow$ Norm($State.Loads$)
    $x_t \leftarrow$ CONCAT(S_a, S_l) ▷ State Encoding
    ▷ Action Selection
    if RANDOM $< \epsilon$ **then**
      $a_t \leftarrow Random(0 \ldots N_{act})$
    else
      $Q \leftarrow Q_Network.Forward(x_t)$
      $a_t \leftarrow$ ArgMax(Q)
    end if
    ▷ Environment Step
    $Next_State, r_t \leftarrow Environment.Step(a_t)$
    STORE($State, a_t, r_t$, Next_State) **in** $\mathcal{D}$
    ▷ Experience Replay
    $Batch \leftarrow Sample(\mathcal{D})$
    $Loss \leftarrow$ MSE($Q_Network(Batch)$, Targets)
    OPTIMISE(Loss)
    $State \leftarrow Next_State$
  end for
  $\epsilon \leftarrow$ DECAY(ϵ)
end for

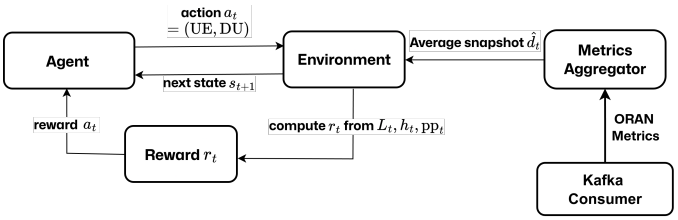


Fig. 2: MDP for traffic steering.

a *no-op*, which happens when the UE stays in the same DU, or re-association of an UE to a different DU. After applying a_t, the environment updates the assignments and recomputes the loads to produce the next state s_{t+1}. The reward r_t penalises load imbalance and the cost of handovers, including ping-pong behaviour.

V. Experimentation and Results

We evaluate the performance of DQL in an experimental laboratory setting with experiments made to validate the user steering policy. The experimental setup is meant to mirror the dynamics of urban networks, which exhibit strongly correlated spatial and temporal traffic patterns, such as commuting peaks, stadium events and the increase of tourism.

In such deployments, high-capacity bands (e.g., n78) often become congested, while lower-frequency bands (e.g., n3) remain underutilised. To simulate these conditions, our study focuses on a scenario where xApps use real-time PRB usage KPM to build a snapshot of the state of the network, and the DQL agent will process this data to reassign the UE, reducing DU load variance.

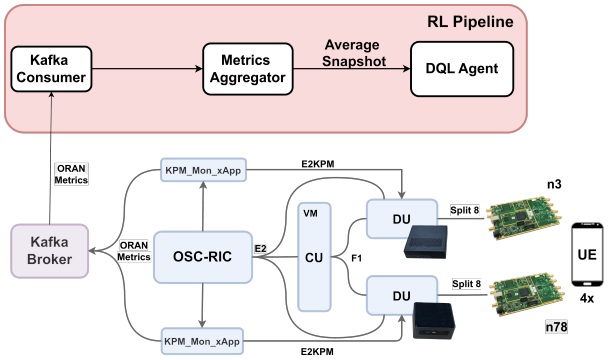


Fig. 3: Real testbed for evaluating the DQL-based load balancing agent in ORAN.

To recreate the aforementioned architecture seen in Figure 1, a testbed using the srsRAN[1] software stack for the disaggregated RAN components was deployed (Figure 3) with one CU hosted in a Virtual Machine (VM) attached to two DUs, with coverage areas overlapping each other: one hosted by an Intel NUC, operating in the n78 band, and the other by a Minisforum miniPC operating in the n3 band, both attached to SDR devices (USRP B210). The CU connects to Open5GS[2] AMF and UPF functions via the N2 and N3 interfaces, respectively. To generate traffic load, four smartphones were used (two Xiaomi Redmi 9T, one Google Pixel 6a and one Google Pixel 7), initially connected to the DU operating in the n78 band, running iperf3[3] clients with Termux[4] producing uplink traffic with different data rates so they have different PRB requirements. For the Near-RT RIC, we used the O-RAN SC RIC running in a second VM with two KPM monitoring xApps from the srsRAN Project repository[5] to subscribe to KPM indicators and collect downlink and uplink PRB usage per-UE. These xApps publish these metrics to a Kafka Broker[6], and then a Kafka Consumer forwards the metrics to a Metrics Aggregator, which computes the average use of PRBs in a 60 second time window used as an input for the DQL pipeline. The DQL traffic steering agent receives the data from the aggregator and then applies simulated re-associations to avoid disrupting user traffic.

A. Initial Configuration Performance

We start from a highly imbalanced initial condition in which all four UEs are associated with the n78 band DU. Using the reference configuration in Table II, the agent performs one decision per step, choosing either a *no-op* or a *handover*. The resulting load evolution for this initial configuration is presented in Figure 4.

In Figure 4a, the different UEs show different PRB demands due to their different offered bitrates (summarised in Table I),

[1] https://www.srsran.com
[2] https://open5gs.org/
[3] https://iperf.fr
[4] https://termux.dev
[5] https://github.com/srsran
[6] https://kafka.apache.org

TABLE I: Smartphones occupied bandwidth and average PRB usage per DU.

UE	Device	Occupied bandwidth (kHz)	Avg PRBs in n78	Avg PRBs in n3
UE1	Xiaomi	3600	20	40
UE2	Xiaomi	3420	18	26
UE3	Google Pixel	1440	8	16
UE4	Google Pixel	900	5	10

TABLE II: Initial Configuration RL and runtime settings.

Parameter	Value
Episodes $\times$ steps/episode	600×50 (30000 decisions)
Discount γ	0.9
Optimizer / LR	Adam / 10^{-3}
Batch size / Replay capacity	32 / 5000
Exploration ε	$1.0 \rightarrow 0.1$ (decay 0.995)
Reward weights (ω, α, β)	$(6, 1, 5)$
Snapshot window	60 s moving average
Action semantics	no-op or handover

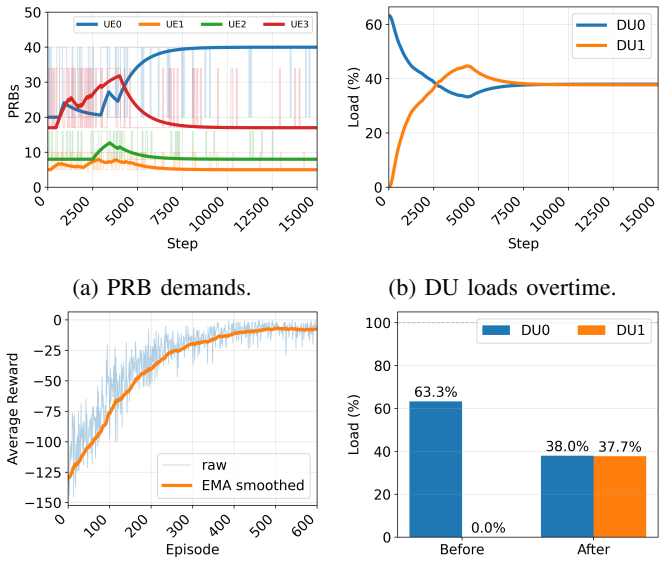
TABLE III: Comparison of Epsilon Decay Schedules.

Schedule	Decay	Stability
Linear	Constant reduction throughout training.	$\approx$ 7500 steps
Exponential	Rapid initial decay followed by slower reduction.	$\approx$ 6000 steps
Cosine	Slow start with accelerated decay mid-training.	$\approx$ 8000 steps
Flat-then-drop	Maintains $\epsilon \approx 1.0$ for the first third of training.	$\approx$ 15000 steps
Per-episode (0.995)	Smooth decay over the full budget.	$\approx$ 8000 steps

TABLE IV: Training budgets explored and total interactions.

Setting	Episodes (E)	Steps/eps (S)	Total steps (N)
Initial Configuration	600	50	30000
Budget A	450	35	15750
Budget B	300	20	6000

and these demands change accordingly when a handover occurs. The UEs consuming more resources are the first to change to the other DU as the agent is still in the exploration phase. A stable state of the network can be seen in Figure 4b at around 8000 steps, where fewer handovers happen, and the agents take fewer necessary decisions. In Figure 4c, the rewards increase roughly from -125 to 0, and then drop off after approximately 500 episodes. Early in training, ε is high, and the agent executes handovers that reduce DU load variance. Those moves raise the per-step reward and the cumulative return, which explains the steep initial slope. Around 150 episodes, the DU loads already sit close to a balanced split, yet the return keeps drifting upward while ε approaches its floor near episode 460, because exploration-driven handovers become rarer and mobility penalties fade. Once ε is clamped and both the target network and the replay dynamics settle, the policy spends most steps in no-op. Per-episode rewards hover near zero with only small fluctuations caused by traffic variability, so the episode return approaches a ceiling. The plateau indicates that the imbalance is minimised and unnecessary handovers are suppressed. In the next subsections, the initial configuration parameters will be changed to find out if a better combination can be achieved.



(a) PRB demands. (b) DU loads overtime.

(c) Rewards. (d) Load comparison

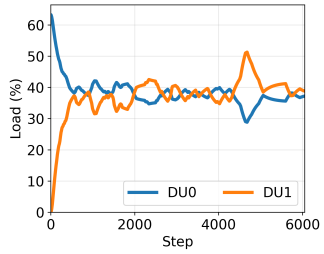
Fig. 4: Initial Configuration Scenario results.

B. Exploration Schedule Evaluation

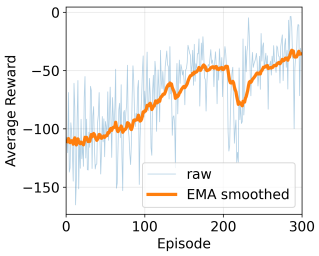
This section details the different types of decay of the parameter ε as shown in Table III. All methods converge to a similar steady state near 38% per DU; hence, the evaluation will be based on efficiency in balancing the DUs. After these tests, we conclude that the best decays are Linear and Exponential (per step), compared to the initial configuration. The linear decay may have a constant reduction with a predictable cadence, but the exponential decay shows the shortest time to balance the network across all tests, meaning that it has the best balance between exploration and exploitation. Bearing this in mind, the exponential decay will be chosen to improve the initial configuration results.

C. Exploring Different Episode Budgets

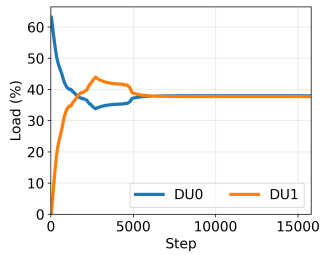
Different episode budgets are defined in Table IV to check if it is able to reduce the number of decisions to reach a stable outcome, and to isolate the effect of the budget. The ε-greedy exploration is kept scheduled by total steps, so that the same exploration limit is reached after a comparable number of interactions rather than episodes. By looking at Figure 5, with 6000 interactions, the Exponential Moving Average (EMA) reward curve shows some improvement from roughly -150 to -35 across the 300 episodes; by looking at the raw reward



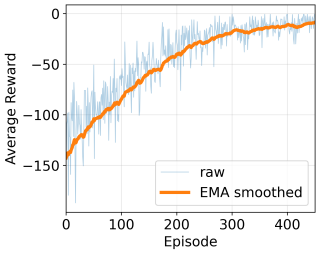
(a) DU Loads with 6000 decisions.



(b) Rewards 6000 decisions.



(c) DU Loads with 15750 decisions.



(d) Rewards with 15750

Fig. 5: Testing training budget.

TABLE V: Reward weight configurations.

Setting	α	β	ω
Initial Configuration	1.00	5.00	6.00
Handover-inducing	0.25	2.00	6.00
Handover-discouraging	3.00	8.00	6.00
Fairness-centric	1.00	5.00	10.00

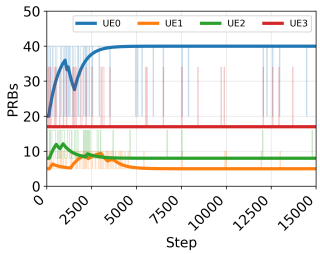
values, it still maintains some inconsistency, meaning that the agent still takes some unnecessary decisions even though the load reached an even balance. With 15750 interactions, learning is more stable; by looking at the EMA reward curve, it is assumed that the agent reaches a plateau at around 350 episodes, meaning that the agent stops making unnecessary decisions and the loads stabilise at around 37%–38%. Moreover, loads reach a balance at around 5000 interactions compared to the initial configuration 8000. Comparing these two budgets with the original one with 30000 decisions, it is assumed that the budget 15750 improves the initial configuration, because it reached the same stable outcome with almost half of the steps. The Budget B is not considered because it does not reach a stable final state. This will lead to the use of the budget of 15750 decisions to improve the agent.

D. Comparing different Reward Weights

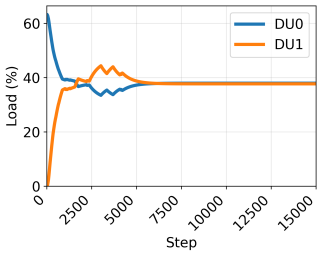
To analyse the impacts of the different reward functions, other reward weights are defined in Table V. The handover-inducing configuration encourages the agent to take earlier action, in terms of the handovers, which leads to a higher variance early on during training. The handover-discouraging configuration produces the smoothest trajectories, but with a slight skew. Finally, the fairness-centric approach marginally improves the final imbalance, offering little clear practical benefit. Given that our primary objective is to balance the antennas' load without materially affecting service, and the observed effect sizes between settings are small under this constraint, as can be seen in Table VI, and since the different configurations reach stability at ≈ 8000, the initial configuration is kept for the remainder of the evaluation.

TABLE VI: Comparison of Reward Design Variants.

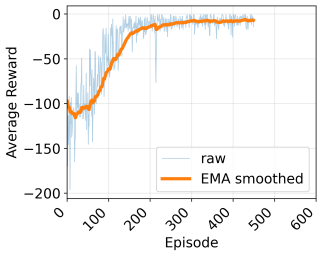
Configuration	Learning	Steps to Balance	Operational Trade-off
Handover-Inducing	Higher variability during early training.	≈ 8000	Encourages early actions.
Handover-Discouraging	Lower variance throughout learning.	≈ 8000	Minimises reassignment overhead.
Fairness-Centric	Rapid improvement in return.	≈ 8000	Maximises load equality while maintaining stability.



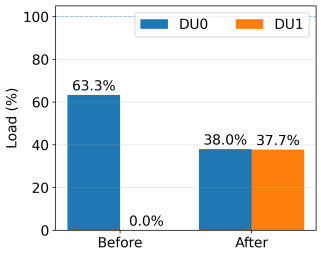
(a) PRB demands.



(b) DU loads overtime.



(c) Rewards.



(d) Load comparison.

Fig. 6: Improved Scenario results.

E. Improving the Initial Configuration

The optimised setting using the reduced training budget and the exponential decay per step results are presented in Figure 6, which demonstrates superior convergence and stability. In this configuration, the UE assignments settle after approximately 5500 steps, after which handovers happen less often. The reward curve exhibits steady growth, levelling off at around -10 by episode 350, compared to the initial configuration that was at 450 episodes; the loads converge continually at around 6000 steps compared to 8000 steps in the initial configuration. We can clearly observe that the original initial configuration has been improved.

VI. Conclusions and Future Work

This paper proposed and experimentally validated a RL-based approach for traffic steering in ORAN deployments. By leveraging standard E2-interface KPM telemetry and the Near-RT RIC platform, the proposed DQL agent effectively balances radio resource loads across overlapping cells, while explicitly penalising unnecessary handovers and ping-pong effects to preserve connection stability. The problem was formulated as a constrained control task that proposes deliberate steering actions and then holds a stable state. PRB loads were normalised to percentages so that cells with mixed numerologies remain comparable. The experimental testbed mirrored a realistic deployment, consisting of a CU, two DUs, and real UEs. Evaluation focused on two practical outcomes: reducing load imbalance and avoiding unnecessary handovers, and the results have shown that the telemetry-driven RL control is feasible in an ORAN Near-RT RIC closed loop, achieving load balancing across overlapping cells even under a highly imbalanced scenario ($63\% - 0\%$ load split).

Future efforts consider: the transitioning from simulated re-associations to actual handovers, incorporating richer metrics to evaluate if the handover is costly; scaling the solution to a multi-cell cluster; and exploring the PRB capping or slicing constraints through E2SM-RC to enforce resource limits per cell.

Acknowledgments

This work was supported in part by the EU's HE research and innovation programme HORIZON-JU-SNS-2023 under the 6G-PATH project (Grant No. 101139172). The views expressed in this contribution are those of the authors and do not necessarily represent the project nor the Commission.

This study was funded by the PRR — Plano de Recuperação e Resiliência and by the NextGenerationEU funds at Universidade de Aveiro, through the scope of the Agenda for Business Innovation "ATT — Agenda Mobilizadora Acelerar e Transformar o Turismo" (Project no. 47 with the application C645192610-00000060).

References

[1] M. Tayyab *et al.*, "A survey on handover management: From lte to nr," *IEEE Access*, vol. 7, pp. 118 907–118 930, 2019.

[2] E. Gures *et al.*, "A comprehensive survey on mobility management in 5g heterogeneous networks: Architectures, challenges and solutions," *IEEE Access*, vol. 8, pp. 195 883–195 913, 2020.

[3] A. Lacva *et al.*, "Programmable and customized intelligence for traffic steering in 5g networks using open ran architectures," *IEEE Transactions on Mobile Computing*, vol. 23, no. 4, p. 2882–2897, Apr. 2024. [Online]. Available: https://doi.org/10.1109/TMC.2023.3266642

[4] M. Polese *et al.*, "Colo-ran: Developing machine learning-based xapps for open ran closed-loop control on programmable experimental platforms," *IEEE Transactions on Mobile Computing*, vol. 22, no. 10, p. 5787–5800, 2023. [Online]. Available: https://doi.org/10.1109/TMC.2022.3188013

[5] ——, "Understanding o-ran: Architecture, interfaces, algorithms, security, and research challenges," *IEEE Communications Surveys Tutorials*, vol. 25, no. 2, pp. 1376–1411, 2023.

[6] P. Ramesh *et al.*, "User association-based load balancing using reinforcement learning in 5G heterogeneous networks," *The Journal of Supercomputing*, vol. 81, no. 1, p. 328, 2024. [Online]. Available: https://doi.org/10.1007/s11227-024-06788-1

[7] P. E. Iturria-Rivera *et al.*, "Qos-aware load balancing in wireless networks using clipped double q-learning," in *2021 IEEE 18th International Conference on Mobile Ad Hoc and Smart Systems (MASS)*, 2021, pp. 10–16.

[8] R. Baker *et al.*, "Real: Reinforcement learning-enabled xapps for experimental closed-loop optimization in o-ran with osc ric and srsran," in *2025 IEEE International Conference on Communications Workshops (ICC Workshops)*. IEEE, Jun. 2025, p. 389–395. [Online]. Available: http://dx.doi.org/10.1109/ICCWorkshops67674.2025.11162144